

preprocessing step used before testing a new image with your trained ResNet18 model.

```
from PIL import Image
from torchvision import transforms

def preprocess_image(image_path):
    image = Image.open(image_path).convert("RGB")
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
    return transform(image).unsqueeze(0) # Add batch dimension
```

=====

This heading and code are used in a **Deep Learning Image Classification project** (like your ResNet18 project).

Heading

"Write a function to load an image and preprocess it for inference."

Meaning

- **Load an image** → Open the image from a folder.
- **Preprocess** → Convert the image into the format required by the trained model.
- **Inference** → Prediction/testing using a trained model.

In simple words:

👉 "Create a function that takes an image, prepares it correctly, and sends it to the model for prediction."

Code Explanation

```
from PIL import Image
```

```
from torchvision import transforms
```

Line 1

```
from PIL import Image
```

Imports the PIL library.

Used to open image files like:

```
image = Image.open("cat.jpg")
```

Line 2

```
from torchvision import transforms
```

Imports image transformation utilities.

Used for:

- Resize
 - Convert image to tensor
 - Normalize
-

Function Definition

```
def preprocess_image(image_path):
```

Creates a function named:

```
preprocess_image()
```

Input:

```
image_path
```

Example:

```
preprocess_image("dog.jpg")
```

Open Image

```
image = Image.open(image_path).convert("RGB")
```

What happens?

1. Opens image file

```
Image.open(image_path)
```

2. Converts image to RGB format

```
.convert("RGB")
```

Example:

Before	After
PNG with transparency	RGB
Grayscale	RGB
RGBA	RGB

Most pretrained models expect **3 channels (RGB)**.

Create Transform Pipeline

```
transform = transforms.Compose([
```

Combines multiple preprocessing steps into one pipeline.

Step 1: Resize

```
transforms.Resize((224,224))
```

Changes image size to:

224 × 224

Example:

Original	After Resize
1024×768	224×224
500×500	224×224

ResNet18 expects 224×224 images.

Step 2: Convert to Tensor

`transforms.ToTensor()`

Converts image to tensor.

Before:

Pixel values:

0 to 255

After:

0.0 to 1.0

Example:

Pixel

255 → 1.0

128 → 0.50

0 → 0.0

Step 3: Normalize

```
transforms.Normalize(  
    [0.485,0.456,0.406],  
    [0.229,0.224,0.225]  
)
```

Normalizes RGB channels.

These are ImageNet dataset values.

Since ResNet18 was pretrained on ImageNet, we use the same normalization.

Mean Values

[0.485,0.456,0.406]

For:

- Red
 - Green
 - Blue
-

Standard Deviation

[0.229,0.224,0.225]

For:

- Red
 - Green
 - Blue
-

Close Transform Pipeline

])

All preprocessing steps are stored in:

transform

Apply Transform

transform(image)

Processes image through:

1. Resize
 2. Tensor conversion
 3. Normalize
-

Add Batch Dimension

.unsqueeze(0)

Very important.

Without unsqueeze

Shape:

[3,224,224]

Model expects:

[batch_size, channels, height, width]

So:

[1,3,224,224]

is required.

The 1 means:

1 image

Return Processed Image

`return transform(image).unsqueeze(0)`

Returns the final tensor ready for prediction.

Complete Flow

Image File



Open Image



Convert RGB



Resize (224×224)



Convert Tensor



Normalize



Add Batch Dimension



Ready for ResNet18 Prediction

Example Usage

```
img_tensor = preprocess_image("dog.jpg")
```

```
output = model(img_tensor)
```

This is exactly the preprocessing step used before testing a new image with your trained **ResNet18 model**.

=====

Code	Purpose	Explanation
<code>from PIL import Image</code>	Import PIL Library	Used to open and work with image files.
<code>from torchvision import transforms</code>	Import Transforms	Used for image preprocessing operations like resize, tensor conversion, and normalization.
<code>def preprocess_image(image_path):</code>	Function Definition	Creates a function that accepts an image path as input.
<code>Image.open(image_path)</code>	Open Image	Loads the image from the specified location.
<code>.convert("RGB")</code>	Convert to RGB	Ensures the image has 3 color channels (Red, Green, Blue).
<code>transforms.Compose([...])</code>	Create Pipeline	Combines multiple preprocessing steps into one sequence.
<code>transforms.Resize((224,224))</code>	Resize Image	Changes image size to 224×224 pixels, required by ResNet18.
<code>transforms.ToTensor()</code>	Convert to Tensor	Converts image into PyTorch tensor format and scales pixel values from 0-255 to 0-1.

<code>transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])</code>	Normalize Image	Applies ImageNet mean and standard deviation values used during ResNet training.
<code>transform(image)</code>	Apply Transformations	Executes all preprocessing steps on the image.
<code>.unsqueeze(0)</code>	Add Batch Dimension	Changes shape from <code>[3, 224, 224]</code> to <code>[1, 3, 224, 224]</code> .
<code>return transform(image).unsqueeze(0)</code>	Return Processed Image	Returns the final image tensor ready for prediction.

=====